



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

قسم: الذكاء الصناعي - السنة الرابعة

وظيفة مقرر البرمجة المتوازية 2026

إعداد الطلاب:

- دعاء حسن يونس
- محمد غزوان ديب
- سيدرا منذر هلال
- سدره جهاد كامل
- محمد قاسم التنبير

إشراف المهندس: حذيفة عقيل

مقدمة

يهدف هذا المشروع إلى تطوير نظام متكامل لمنصة تجارة إلكترونية يتمتع بقدرة عالية على معالجة أعداد كبيرة من الطلبات المتزامنة بكفاءة وموثوقية. ويركز المشروع بصورة أساسية على تطبيق المتطلبات غير الوظيفية (Non-Functional Requirements)، مثل تحسين الأداء، وضمان سلامة البيانات ومنع تعارض الوصول إليها، وإدارة الموارد الحاسوبية بكفاءة، وأتمتة العمليات الخلفية، أكثر من تركيزه على تنوع الوظائف والخدمات التي يقدمها النظام.

ولتحقيق هذه الأهداف، تم الاعتماد على إطار العمل **Django** إلى جانب **Django REST Framework** لتطوير واجهات برمجة التطبيقات (REST APIs)، كما استُخدم **Redis** ليؤدي دورين رئيسيين يتمثلان في التخزين المؤقت (Caching) والعمل كوسيط للرسائل (Message Broker)، بينما استُخدمت مكتبة **Celery** لتنفيذ المهام الخلفية بصورة غير متزامنة، واعتمد النظام على **SQLite** كقاعدة البيانات الأساسية لتخزين وإدارة البيانات.

يركز هذا التقرير على توثيق كيفية تطبيق المتطلبات غير الوظيفية العشرة المطلوبة داخل المشروع، حيث يتناول كل مطلب من خلال ثلاثة محاور رئيسية. يبدأ المحور الأول بعرض الأساس النظري للمفهوم وبيان أهميته في الأنظمة البرمجية الحديثة، ثم ينتقل إلى توضيح آلية تطبيق هذا المفهوم عملياً داخل النظام باستخدام أمثلة حقيقية من الشيفرة البرمجية المنفذة، ويختتم بعرض نتائج الاختبارات العملية التي أجريت للتحقق من صحة التطبيق وإثبات كفاءة النظام أثناء التشغيل الفعلي.

ويستند هذا التقرير إلى نتائج عملية تم الحصول عليها من تشغيل النظام واختباره في بيئة تنفيذ حقيقية، مما يضمن أن جميع النتائج والاستنتاجات الواردة فيه مبنية على تطبيق فعلي، وليست مجرد افتراضات أو مناقشات نظرية.

1. حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

يُعد تعارض الوصول المتزامن (Race Condition) من أكثر المشكلات شيوعاً وخطورة في الأنظمة التي تسمح بتنفيذ عدد كبير من العمليات في الوقت نفسه، حيث تنشأ هذه المشكلة عندما يحاول أكثر من مستخدم أو عملية الوصول إلى المورد نفسه وتعديله بشكل متزامن. وفي هذه الحالة قد تعتمد إحدى العمليات على بيانات لم تعد تمثل الحالة الحالية للمورد، مما يؤدي إلى تنفيذ تحديثات غير صحيحة أو الكتابة فوق تعديلات أجرتها عملية أخرى.

وتبرز هذه المشكلة بصورة واضحة في أنظمة التجارة الإلكترونية، وبشكل خاص عند إدارة المخزون. فعلى سبيل المثال، إذا كانت الكمية المتبقية من أحد المنتجات تساوي وحدة واحدة فقط، ثم حاول مستخدمان شراء هذه الوحدة في اللحظة نفسها، فقد يقرأ كلا الطلبين قيمة المخزون قبل أن يتم تحديثها، فيعتقد كل منهما أن المنتج لا يزال متوفراً، وينتج عن ذلك إتمام عمليتي بيع للوحدة نفسها، مما يؤدي إلى ظهور قيمة سالبة للمخزون أو تسجيل بيانات غير صحيحة داخل قاعدة البيانات.

ولمنع حدوث هذا النوع من التعارض، يجب توفير آلية تضمن تنفيذ عمليات التعديل بصورة آمنة، بحيث لا يُسمح إلا لعملية واحدة بتعديل المورد في اللحظة نفسها، مع التحقق من صحة البيانات قبل تنفيذ أي عملية كتابة فعلية. ولهذا السبب اعتمد المشروع على مزيج من القيود البنوية التي تفرضها قاعدة البيانات، بالإضافة إلى التحقق البرمجي داخل منطق الأعمال، بحيث يعمل هذان المستويان معاً لضمان سلامة البيانات والمحافظة على اتساقها حتى في حالات الضغط والتزامن المرتفع.

التطبيق في المشروع

في نموذج البيانات (**models.py**) تم تعريف حقل كمية المخزون باستخدام النوع **PositiveIntegerField**، والذي يفرض على مستوى قاعدة البيانات أن تكون قيمة المخزون عدداً صحيحاً موجباً، وبالتالي يمنع تخزين أي قيمة سالبة داخل قاعدة البيانات حتى في حال حدوث خطأ برمجي غير متوقع.

كما أضيف الحقل **version** الذي يُستخدم لتتبع عدد مرات تعديل المنتج، حيث يتم زيادة قيمته مع كل عملية تحديث ناجحة، الأمر الذي يساعد على تتبع التعديلات وإدارة العمليات المتعلقة بتزامن الوصول.

```
class Product(TimeStampedModel):
```

```
    stock_quantity = models.PositiveIntegerField()
```

```
    version = models.PositiveIntegerField(default=0)
```

أما داخل طبقة الخدمات (**services.py**) فقد جرى تنفيذ آلية التحقق المنطقي قبل تعديل كمية المخزون. إذ تُنفذ العملية كاملة داخل معاملة ذرية (**transaction.atomic()**) لضمان أن جميع خطواتها تتم كوحدة واحدة، كما يُستخدم الأمر **select_for_update()** لقفل سجل المنتج ومنع أي عملية أخرى من تعديله حتى انتهاء العملية الحالية.

بعد الحصول على القفل، يتم حساب الكمية الجديدة للمخزون، ثم التحقق من أنها لن تصبح سالبة بعد تنفيذ عملية الخصم أو الإضافة. وفي حال كانت النتيجة أقل من الصفر، يُرفع الاستثناء **StockUpdateError** ويتم إلغاء العملية بالكامل، أما إذا كانت الكمية صحيحة، فيُحدَّث المخزون وتزداد قيمة الحقل **version** ثم تُحفظ التعديلات في قاعدة البيانات.

```
def adjust_stock(product_id, change):
```

```
    with transaction.atomic():
```

```
        product = Product.objects.select_for_update().get(id=product_id)
```

```
        new_quantity = product.stock_quantity + change
```

```
        if new_quantity < 0:
```

```
            raise StockUpdateError("Stock cannot become negative")
```

```
        product.stock_quantity = new_quantity
```

```
        product.version += 1
```

```
        product.save(update_fields=["stock_quantity", "version"])
```

يضمن هذا الأسلوب أن جميع عمليات تعديل المخزون تتم بصورة متسلسلة وآمنة، بحيث لا يمكن لطلبين متزامنين تعديل السجل نفسه في الوقت ذاته، كما يمنع حفظ أي قيمة غير صحيحة داخل قاعدة البيانات حتى عند تعرض النظام لعدد كبير من الطلبات المتزامنة.

نتائج الاختبار الفعلي

للتحقق من كفاءة هذه الآلية، أُجري اختبار عملي باستخدام منتج لا يحتوي إلا على وحدة واحدة متبقية في المخزون، ثم نُفذت محاولة شراء بكمية مقدارها وحدتان، أي أكثر من الكمية المتاحة فعلياً.

أثناء تنفيذ العملية قام النظام بفحص كمية المخزون قبل الخصم، واكتشف أن الكمية المطلوبة تتجاوز الكمية المتوفرة، ولذلك رفع الاستثناء **OutOfStockError** ومنع إكمال عملية الشراء.

وبعد انتهاء الاختبار، تم إجراء فحص مباشر لقاعدة البيانات للتأكد من حالة المنتج، حيث تبين أن قيمة المخزون بقيت كما هي دون أي تعديل، ولم تُسجل أي قيمة سالبة أو بيانات غير صحيحة. وتؤكد هذه النتيجة نجاح الآلية المطبقة في حماية البيانات المشتركة ومنع حدوث مشكلات تعارض الوصول المتزامن (Race Condition)، حتى عند محاولة تنفيذ عمليات شراء غير صحيحة أو متزامنة.

2. إدارة الموارد الحاسوبية (Resource Management & Capacity Control)

تُعد إدارة الموارد الحاسوبية من المتطلبات الأساسية في الأنظمة عالية الأداء، إذ تعتمد قدرة النظام على الاستمرار في العمل بكفاءة على كيفية استغلال موارده المتاحة، مثل الذاكرة الرئيسية، ووحدة المعالجة المركزية، واتصالات قاعدة البيانات، والخيوط البرمجية (Threads). وعند غياب آلية فعالة لإدارة هذه الموارد، فإن استقبال عدد كبير من الطلبات خلال فترة زمنية قصيرة قد يؤدي إلى استنزاف موارد الخادم بالكامل، الأمر الذي ينعكس على انخفاض الأداء، وارتفاع أزمّة الاستجابة، وقد يصل في بعض الحالات إلى توقف الخدمة بشكل كامل.

وفي المقابل، فإن وضع قيود صارمة أكثر من اللازم على عدد العمليات المتزامنة قد يؤدي إلى تقليل الاستفادة من إمكانيات الخادم، وبالتالي انخفاض الإنتاجية دون مبرر. لذلك فإن الهدف من إدارة الموارد ليس تقليل عدد الطلبات، وإنما تحقيق توازن يضمن استغلال الموارد بكفاءة مع الحفاظ على استقرار النظام.

ولتحقيق هذا التوازن، تعتمد العديد من الأنظمة الحديثة على آليات تعرف باسم **Capacity Control**، والتي تقوم بتحديد عدد أقصى من العمليات التي يمكن تنفيذها في الوقت نفسه. وعند وصول طلبات جديدة بعد امتلاء السعة المتاحة، يتم رفضها مؤقتاً برسالة واضحة بدلاً من السماح بتراكمها بصورة قد تؤدي إلى انهيار النظام.

التطبيق في المشروع

لتطبيق هذا المفهوم، تم إنشاء طبقة وسيطة (Middleware) تعمل قبل وصول الطلب إلى منطق الأعمال، بحيث تتحكم في عدد الطلبات التي يسمح للنظام بمعالجتها في الوقت نفسه.

اعتمدت هذه الطبقة على الكائن **BoundedSemaphore** من مكتبة **threading**، والذي يعمل على حجز عدد محدد من "المقاعد" المتاحة لمعالجة الطلبات. وعند وصول طلب جديد، يحاول النظام الحصول على أحد هذه المقاعد، فإذا نجح استمرت معالجة الطلب بصورة طبيعية، أما إذا كانت جميع المقاعد مشغولة، فإن الطلب ينتظر مدة زمنية محددة، وفي حال عدم توفر سعة خلال هذه المدة يعاد للمستخدم رد يفيد بأن الخادم مشغول حالياً.

```
_request_slots = threading.BoundedSemaphore(
```

```
settings.MAX_CONCURRENT_REQUESTS
```

```
)
```

```

class CapacityControlMiddleware:

    def __call__(self, request):

        acquired = _request_slots.acquire(
            timeout=settings.CAPACITY_WAIT_SECONDS
        )

        if not acquired:

            return JsonResponse(
                {"error": "server is busy"},
                status=503
            )

        try:

            return self.get_response(request)

        finally:

            _request_slots.release()

```

كما جرى تحديد الحد الأقصى لعدد الطلبات المتزامنة داخل ملف **settings.py** من خلال المتغير **MAX_CONCURRENT_REQUESTS**، والذي صُيِّط على خمسين طلباً متزامناً، وهي قيمة تحقق توازناً مناسباً بين استغلال موارد الخادم والحفاظ على استقرار النظام أثناء التشغيل.

نتائج الاختبار الفعلي

للتحقق من فعالية هذه الآلية، أُجري اختبار ضغط باستخدام مئة مستخدم افتراضي يعملون بصورة متزامنة، حيث أرسلت جميع الطلبات إلى النظام في الوقت نفسه لمحاكاة ظروف التشغيل الواقعية.

أظهرت نتائج الاختبار أن الخادم حافظ على استقراره طوال مدة الاختبار التي بلغت ستين ثانية، ولم يحدث أي توقف للخدمة أو تجمد في معالجة الطلبات، على الرغم من الارتفاع الكبير في عدد العمليات المتزامنة.

كما أثبتت النتائج أن طبقة التحكم بالسعة أدت دورها بنجاح، إذ منعت استنزاف موارد الخادم، واستمرت جميع الطلبات المقبولة في المعالجة بصورة طبيعية، بينما كانت الطلبات التي تجاوزت الحد المسموح به تتلقى استجابة مناسبة دون التأثير في بقية المستخدمين أو في استقرار النظام.

3. المعالجة غير المتزامنة (Asynchronous Queues)

تحتوي تطبيقات التجارة الإلكترونية على العديد من العمليات التي لا يحتاج المستخدم إلى انتظار انتهائها قبل استلام استجابة الطلب، مثل إرسال رسائل البريد الإلكتروني، وإصدار الفواتير، وتسجيل البيانات التحليلية، وإرسال الإشعارات. وإذا نُفذت هذه العمليات داخل مسار الطلب الرئيسي بطريقة متزامنة (Synchronous)، فإن زمن استجابة النظام سيزداد بصورة ملحوظة، رغم أن هذه العمليات لا تؤثر بشكل مباشر في نجاح عملية الشراء نفسها.

وللتغلب على هذه المشكلة، تعتمد الأنظمة الحديثة على المعالجة غير المتزامنة (Asynchronous Processing)، حيث تُرسل هذه العمليات إلى طابور مهام (Task Queue)، بينما يعيد الخادم الاستجابة للمستخدم مباشرة. بعد ذلك، تتولى عمليات مستقلة تعرف باسم **Workers** تنفيذ هذه المهام في الخلفية دون التأثير على زمن استجابة النظام.

التطبيق في المشروع

اعتمد المشروع على مكتبة **Celery** لتنفيذ المهام الخلفية، مع استخدام **Redis** كوسيط للرسائل بين التطبيق وعمال التنفيذ.

بدأت عملية الإعداد من خلال تهيئة تطبيق **Celery** وربطه بإعدادات **Django**، مع تمكين الاكتشاف التلقائي لجميع المهام المعرفة داخل المشروع.

```
os.environ.setdefault(
```

```
"DJANGO_SETTINGS_MODULE",
```

```
"commerce_engine.settings"
```

```
)
```

```
app = Celery("commerce_engine")
```

```
app.config_from_object(
```

```
"django.conf.settings",
```

```
namespace="CELERY"
```

```
)
```

```
app.autodiscover_tasks()
```

بعد ذلك جرى تعريف المهام الخلفية داخل ملف **tasks.py**، بحيث تدعم إعادة المحاولة تلقائياً في حال حدوث خطأ مؤقت أثناء التنفيذ، مع تطبيق أسلوب **Exponential Backoff** لتأخير المحاولات اللاحقة بصورة تدريجية.

```
@shared_task(
```

```

bind=True,

autoretry_for=(Exception,),

retry_backoff=True,

retry_kwargs={"max_retries": 3},

)

def send_order_confirmation(self, order_id: int):

    order = Order.objects.get(id=order_id)

    time.sleep(1)

    return {

        "order_id": order.id,

        "email": order.customer_email,

        "sent": True,

    }

```

وبدلاً من تنفيذ هذه العمليات مباشرة داخل منطق إنشاء الطلب، أصبحت تُرسل إلى طابور Celery باستخدام الدالة **delay()**، مما يسمح للنظام بإكمال عملية الشراء وإرجاع الاستجابة للمستخدم فوراً، بينما تُنفذ العمليات الأخرى في الخلفية.

```
(send_order_confirmation.delay(order.id
```

```
(generate_invoice.delay(order.id
```

```
(log_order_analytics.delay(order.id
```

نتائج الاختبار الفعلي

بعد تشغيل عامل Celery بصورة فعلية، ظهرت جميع المهام المعرفة داخل المشروع عند بدء تشغيل العامل، مما أكد نجاح عملية التهيئة وربط النظام بطابير المهام.

كما أُجريت عملية شراء حقيقية، وأظهرت سجلات التشغيل أن العامل استقبل المهام الثلاث ونفذها بنجاح، حيث احتوت نتائج التنفيذ على معرف الطلب والبريد الإلكتروني وقيمة الطلب الصحيحة، وجميعها تطابقت مع بيانات عملية الشراء المنفذة.

وفي الوقت نفسه، استلم المستخدم استجابة الطلب مباشرة دون انتظار انتهاء تنفيذ هذه المهام، وهو ما يؤكد نجاح تطبيق مفهوم المعالجة غير المتزامنة وتحقيق الهدف الأساسي منه، والمتمثل في تقليل زمن الاستجابة وتحسين تجربة المستخدم.

4. معالجة البيانات الضخمة على دفعات (Batch Processing)

تُعد معالجة كميات كبيرة من البيانات ضمن طلب HTTP واحد من الأساليب غير المناسبة في الأنظمة عالية الأداء، إذ تؤدي إلى بقاء اتصال المستخدم مفتوحاً لفترة طويلة، واستهلاك موارد الخادم بصورة مستمرة حتى انتهاء جميع العمليات. ويزداد هذا التأثير وضوحاً عند تنفيذ عمليات مثل جرد المبيعات اليومية، أو تحديث آلاف السجلات، أو إنشاء التقارير الدورية، حيث قد يتجاوز زمن التنفيذ الحدود الزمنية المسموح بها، مما يؤدي إلى انتهاء الاتصال قبل اكتمال العملية.

وللتغلب على هذه المشكلة، تعتمد الأنظمة الحديثة على مفهوم **Batch Processing**، والذي يقوم على نقل العمليات طويلة التنفيذ إلى مهام تعمل في الخلفية، بحيث تُقسَّم البيانات إلى دفعات يمكن معالجتها بصورة مستقلة، دون التأثير في استجابة النظام أو في تجربة المستخدم.

التطبيق في المشروع

لتطبيق هذا المفهوم، تم إنشاء مهمة خلفية داخل **tasks.py** تتولى معالجة جميع الطلبات غير المعالجة. تبدأ المهمة بالحصول على سجل الدفعة، ثم تجلب جميع الطلبات التي لم تتم معالجتها، وبعد ذلك تعالج كل طلب على حدة حتى اكتمال الدفعة بالكامل، ثم تُحدَّث حالة الدفعة إلى **completed**.

```
@shared_task
```

```
def process_sales_batch_task(job_id: int) -> str:
```

```
    job = BatchJob.objects.get(id=job_id)
```

```
    unprocessed_orders = list(
```

```
        Order.objects.filter(is_processed=False)
```

```
    )
```

```
    # acquire distributed lock
```

```
    for order in unprocessed_orders:
```

```
        order.is_processed = True
```

```
        order.save(update_fields=["is_processed"])
```



```
job.status = "completed"
```

```
job.save(update_fields=["status"])
```

كما جرى إنشاء واجهة إدارية مخصصة (**process_batch_view**) تسمح ببدء عملية المعالجة، حيث يُنشأ سجل جديد داخل جدول **BatchJob** بحالة **pending**، ثم تُرسل المهمة إلى Celery باستخدام:

```
(process_sales_batch_task.delay(job.id
```

وبمجرد بدء التنفيذ، تتغير حالة الدفعة تدريجياً من **pending** إلى **processing** ثم إلى **completed** بعد انتهاء جميع العمليات.

نتائج الاختبار الفعلي

أُجري اختبار ضغط باستخدام مئة مستخدم متزامن، مع تنفيذ عدة عمليات لمعالجة الدفعات في الوقت نفسه. وأظهرت النتائج أن جميع طلبات تشغيل الدفعات ومراقبة حالتها نُفذت بنجاح، ولم تُسجل أي حالات فشل طوال مدة الاختبار، رغم وجود عدة محاولات متزامنة لإنشاء دفعات جديدة.

وتؤكد هذه النتيجة نجاح آلية معالجة البيانات على دفعات في تنفيذ العمليات طويلة المدة دون التأثير في استجابة النظام أو استقراره.

5. توزيع الأحمال (Load Distribution)

تعتمد الأنظمة الموزعة على مبدأ توزيع الأحمال بين عدة عقد معالجة (**Worker Nodes**)، بهدف منع تركيز جميع المهام على خادم واحد، وتحقيق الاستفادة المثلى من الموارد المتاحة. وتوجد العديد من خوارزميات توزيع الأحمال، إلا أن خوارزمية **Round-Robin** تُعد من أبسط وأكثر الخوارزميات استخداماً، حيث تعتمد على توزيع المهام بالتناوب بين جميع العقد، بحيث تحصل كل عقدة على مهمة واحدة قبل أن تبدأ الدورة التالية.

ويتميز هذا الأسلوب بسهولة تنفيذه وانخفاض تكلفته الحسابية، كما يحقق توزيعاً عادلاً عندما تكون المهام متقاربة في زمن التنفيذ.

تبرير اختيار خوارزمية Round-Robin

وقع الاختيار على خوارزمية **Round-Robin** للأسباب التالية:

- سهولة تنفيذها وعدم حاجتها إلى معلومات مسبقة حول الحمل الحالي لكل عقدة.
- انخفاض تكلفتها الحسابية، حيث تعتمد فقط على عملية حساب باقي القسمة، مما يجعلها ذات تعقيد زمني **O(1)**.
- ملاءمتها لطبيعة مهام المشروع، إذ تتشابه عمليات معالجة الطلبات في الزمن اللازم لتنفيذها.
- بساطة اختبارها والتحقق من صحة نتائجها مقارنة بخوارزميات أكثر تعقيداً مثل **Least Connections** أو **Weighted Round-Robin**.

ملاحظة حول نطاق التطبيق

لم يعتمد المشروع على مجموعة خوادم فعلية، وإنما تم تنفيذ محاكاة كاملة لخوارزمية التوزيع نفسها، بحيث يجري توزيع المهام على عقد افتراضية تمثل البيئة الحقيقية، وهو ما يحقق متطلبات المشروع الخاصة بمحاكاة توزيع الأحمال مع المحافظة على دقة التقرير وعدم الادعاء بوجود بنية موزعة فعلية.

التطبيق في المشروع

تم تنفيذ الخوارزمية داخل الملف **load_distribution.py** من خلال إنشاء مجموعة من العقد الافتراضية، ثم بناء مجلد يعتمد على التوزيع الدوري بين هذه العقد.

```
SIMULATED_NODES = [
```

```
    "worker-node-1",
```

```
    "worker-node-2",
```

```
    "worker-node-3",
```

```
]
```

```
class RoundRobinScheduler:
```

```
    def __init__(self, nodes=None):
```

```
        self.nodes = nodes or SIMULATED_NODES
```

```
        self._counter = 0
```

```
    def next_node(self):
```

```
        node = self.nodes[
```

```
            self._counter % len(self.nodes)
```

```
        ]
```

```
        self._counter += 1
```

```
        return node
```

ثم استُخدمت هذه الخوارزمية أثناء معالجة الدفعات، حيث يُحدد لكل طلب العامل المسؤول عن معالجته، مع تسجيل معلومات التوزيع داخل السجل الخاص بالنظام.

```
scheduler = RoundRobinScheduler()
```

```
assignments = distribute_batch(
```

```
    unprocessed_orders,
```

```
    scheduler=scheduler,
```

```
)
```

```
for order, assigned_node in assignments:
```

```
    logger.info(
```

```
        "Batch: processing order %s on %s",
```

```
        order.id,
```

```
        assigned_node,
```

```
)
```

نتائج الاختبار الفعلي

أُجري اختبار باستخدام خمس عشرة مهمة موزعة على ثلاث عقد، وأظهرت النتائج حصول كل عقدة على خمس مهام، وهو توزيع متساوٍ تماماً كما تتوقعه خوارزمية Round-Robin.

كما أُعيد الاختبار باستخدام عشر مهام فقط، وكانت نتيجة التوزيع (4، 3، 3)، وهو أيضاً التوزيع النظري الصحيح، إذ لا يزيد الفرق بين أي عقدتين على مهمة واحدة، مما يؤكد عدالة الخوارزمية ونجاح تنفيذها داخل المشروع.

6. استراتيجية التخزين المؤقت الموزع (Distributed Caching)

تتكرر قراءة بعض البيانات داخل أنظمة التجارة الإلكترونية بصورة كبيرة، مثل قوائم المنتجات أو المنتجات الأكثر مبيعاً، في حين أن معدل تغير هذه البيانات يكون منخفضاً نسبياً. لذلك فإن تنفيذ استعلام جديد إلى قاعدة البيانات في كل مرة يطلب فيها المستخدم هذه المعلومات يؤدي إلى استهلاك موارد إضافية وزيادة زمن الاستجابة دون وجود حاجة حقيقية لذلك.

ولحل هذه المشكلة، تعتمد الأنظمة الحديثة على التخزين المؤقت (Caching)، وبشكل خاص نمط **Cache-Aside**، حيث يُفحص الكاش أولاً، فإذا كانت البيانات موجودة تُعاد مباشرة دون الرجوع إلى قاعدة البيانات، أما إذا لم تكن موجودة فيُنْفَذ الاستعلام، ثم تُخزن النتيجة داخل الكاش لفترة زمنية محددة (TTL)، بحيث تستفيد منها الطلبات اللاحقة.

التطبيق في المشروع

اعتمد المشروع على **Redis** ليعمل كطبقة تخزين مؤقت، حيث جرى تعريف مفتاح خاص بقائمة المنتجات، بالإضافة إلى مدة صلاحية تبلغ ثلاثين ثانية.

```
"PRODUCT_LIST_CACHE_KEY = "products:list
```

```
PRODUCT_LIST_TTL = 30
```

وعند وصول طلب عرض المنتجات، يبدأ النظام بالبحث داخل **Redis**، فإذا وجد البيانات المخزنة يعيدها مباشرة، أما إذا لم يجدها، فإنه يجلب البيانات من قاعدة البيانات، ثم يخزنها داخل **Redis** قبل إرسالها للمستخدم.

```
def get(self, request):
```

```
    cached = cache.get(  
        PRODUCT_LIST_CACHE_KEY  
    )
```

```
    if cached is not None:
```

```
        return Response(cached)
```

```
    products = Product.objects.order_by("id")
```

```
    payload = {
```

```
        "products":
```

```
        ProductSerializer(  
            products,
```

```
        ),
```

```

        many=True

    ).data

}

cache.set(

    PRODUCT_LIST_CACHE_KEY,

    payload,

    timeout=PRODUCT_LIST_TTL

)

return Response(payload)

```

إضافة إلى دوره في التخزين المؤقت، استُخدم Redis أيضاً كوسيط رسائل (**Message Broker**) وكمخزن لنتائج مهام Celery (**Result Backend**)، مما جعله عنصراً أساسياً في أكثر من جزء داخل بنية النظام.

نتائج الاختبار الفعلي

تم التحقق أولاً من نجاح الاتصال بين Django و Redis من خلال تخزين قيمة نصية ثم استعادتها مباشرة، وقد نجحت العملية دون أي أخطاء، مما أكد سلامة الاتصال بين النظام وخادم Redis.

بعد ذلك أُجري اختبار لقياس أثر التخزين المؤقت على زمن الاستجابة، حيث أرسلت خمس طلبات أولية قبل وجود البيانات داخل الكاش، ثم أرسلت خمسون طلباً بعد تخزين البيانات.

وأظهرت النتائج أن متوسط زمن الاستجابة انخفض من **ms 2193.76** إلى **ms 2059.60**، أي بتحسن يقارب **6%**.

ورغم أن هذه النسبة أقل من التحسن النظري المتوقع للتخزين المؤقت، فإنها تؤكد نجاح آلية الكاش في العمل بصورة صحيحة. كما تشير النتائج إلى وجود عامل تأخير آخر داخل بيئة التشغيل يؤثر في زمن الاستجابة، مثل الشبكة المحلية أو إحدى طبقات المعالجة الأخرى، ولذلك جرى توثيق هذه النتيجة كما ظهرت فعلياً حفاظاً على دقة التقرير وموضوعيته.

7. التحكم في الأقفال (Concurrency Control)

يُعد التحكم في الأقفال من أهم الآليات المستخدمة لضمان سلامة البيانات عند تنفيذ عمليات متزامنة على المورد نفسه. فعندما تحاول عدة عمليات تعديل السجل ذاته في الوقت نفسه، يصبح من الضروري وجود آلية تمنع تعارض هذه العمليات وتحافظ على اتساق البيانات داخل قاعدة البيانات.

وتوجد استراتيجيتان رئيسيتان للتحكم بالتزامن. الأولى هي **القفل التشاؤمي (Pessimistic Locking)**، والذي يفترض منذ البداية احتمال حدوث تعارض، ولذلك يقوم بقفل السجل بمجرد قراءته بغرض التعديل، بحيث لا تستطيع أي عملية أخرى الوصول إليه حتى انتهاء العملية الحالية. أما الثانية فهي **القفل المتفائل (Optimistic Locking)**، والذي يسمح لجميع العمليات بقراءة البيانات بحرية، ثم يتحقق عند الحفظ من أن البيانات لم تتغير منذ لحظة قراءتها، وغالباً ما يعتمد في ذلك على مقارنة رقم الإصدار (Version Number).

ويُفضل استخدام القفل التشاؤمي في الأنظمة التي يُتوقع فيها وجود معدل مرتفع من التنافس على البيانات، كما هو الحال في عمليات شراء المنتجات محدودة الكمية، لأنه يمنع حدوث التعارض قبل وقوعه، بينما يُستخدم القفل المتفائل عادةً عندما يكون احتمال التعارض منخفضاً، مما يسمح بتحقيق أداء أعلى.

التطبيق في المشروع

اعتمد المشروع على **القفل التشاؤمي** باعتباره الآلية الأساسية لحماية بيانات المخزون، وذلك باستخدام الدالة **select_for_update()** داخل معاملة ذرية (**transaction.atomic()**)، بحيث يُقفل سجل المنتج طوال مدة تنفيذ عملية التعديل.

with transaction.atomic():

```
product = Product.objects.select_for_update().get(
    id=product_id
)
```

```
if product.stock_quantity < line.quantity:
```

```
    raise OutOfStockError(...)
```

```
product.stock_quantity -= line.quantity
```

```
product.version += 1
```

```
product.save(
```

```
    update_fields=[
```

```
        "stock_quantity",
```

"version"

]

)

كما يحتوي نموذج **Product** على الحقل **version** الذي يزداد تلقائياً مع كل عملية تحديث ناجحة، ويُستخدم حالياً لمتابعة عدد التعديلات التي تُجرى على المنتج، وليس كآلية كاملة لتطبيق القفل المتفائل.

لمنع تنفيذ العملية Redis باستخدام **(Distributed Lock)** وبالإضافة إلى القفل داخل قاعدة البيانات، تم تطبيق **قفل موزع** المنطقية نفسها أكثر من مرة في الوقت ذاته، مثل الضغط المزدوج على زر الدفع أو إرسال الطلب نفسه مرتين.

```
lock_key = f"lock:checkout:{user.id}"
```

```
if not cache.add(
```

```
    lock_key,
```

```
    _checkout_hash(user.id, lines),
```

```
    timeout=CHECKOUT_LOCK_TIMEOUT,
```

```
):
```

```
    raise DistributedLockError(
```

```
        "Checkout already in progress"
```

```
)
```

يساعد هذا القفل على منع إنشاء أكثر من عملية دفع للمستخدم نفسه في اللحظة نفسها، حتى قبل الوصول إلى قاعدة البيانات.

نتائج الاختبار الفعلي

تم اختبار هذه الآلية من خلال إنشاء قفل يدوي يحاكي وجود عملية دفع قيد التنفيذ، ثم أرسلت محاولة ثانية لتنفيذ عملية دفع جديدة للمستخدم نفسه.

أظهرت نتائج الاختبار أن النظام رفض الطلب الثاني مباشرة، وأصدر الاستثناء **DistributedLockError** كما هو متوقع، مما يؤكد نجاح آلية القفل الموزع في اكتشاف العمليات المكررة ومنع تنفيذها، إلى جانب نجاح القفل التشاؤمي في حماية بيانات المخزون من التعديل المتزامن.

8. سلامة المعاملات (ACID / Transaction Integrity)

تتكون العديد من العمليات داخل أنظمة التجارة الإلكترونية من مجموعة خطوات مترابطة يجب تنفيذها كوحدة واحدة. فعملية إنشاء طلب شراء، على سبيل المثال، تتضمن التحقق من توفر المخزون، وخصم الكمية المطلوبة، وإنشاء سجل الطلب، وإضافة عناصر الطلب، وحساب المبلغ الإجمالي، وتحديث حالة الطلب.

ولو فشلت إحدى هذه الخطوات بعد تنفيذ الخطوات السابقة، فإن استمرار التغييرات الجزئية سيؤدي إلى فساد البيانات وظهور حالات غير متسقة داخل النظام.

ولهذا تعتمد قواعد البيانات على مبادئ **ACID**، والتي تتكون من أربعة عناصر رئيسية هي: الذرية (Atomicity)، والاتساق (Consistency)، والعزل (Isolation)، والديمومة (Durability). ويُعد مبدأ الذرية أهم هذه المبادئ في هذا السياق، إذ يضمن تنفيذ جميع خطوات المعاملة بالكامل أو التراجع عنها بالكامل عند حدوث أي خطأ، بينما يضمن العزل عدم تأثير المعاملات المتزامنة في بعضها البعض أثناء التنفيذ.

التطبيق في المشروع

بحيث لا يتم تثبيت أي **transaction.atomic()** جرى تنفيذ جميع خطوات عملية الدفع داخل معاملة ذرية واحدة باستخدام تغيير في قاعدة البيانات قبل نجاح جميع الخطوات.

with transaction.atomic():

```
order = Order.objects.create(
    user=user,
    customer_email=user.email,
    status=Order.Status.PENDING,
)
```

total = 0

for line in lines:

```
product = Product.objects.select_for_update().get(
    id=line.product_id
```


)

```
if product.stock_quantity < line.quantity:
    raise OutOfStockError(
        f"Insufficient stock for {product.name}"
    )
```

```
product.stock_quantity -= line.quantity
```

```
product.save(
    update_fields=[
        "stock_quantity",
        "version",
    ]
)
```

```
OrderItem.objects.create(
    order=order,
    product=product,
    quantity=line.quantity,
    unit_price=product.price,
)
```

```
total += product.price * line.quantity
```

```
order.total_amount = total
```

```
order.status = Order.Status.PAID
```

```
order.save(  
    update_fields=[  
        "total_amount",  
        "status",  
    ]  
)
```

وفي حال فشل أي جزء من العملية، مثل عدم توفر كمية كافية لأحد المنتجات، فإن Django يتراجع تلقائياً عن جميع التعديلات التي أجريت داخل المعاملة، بما في ذلك إنشاء الطلب وتحديث المخزون وإنشاء عناصر الطلب.

نتائج الاختبار الفعلي

للتحقق من صحة تطبيق مبدأ ACID، نُفذ اختباران عمليان.

في الاختبار الأول، تمت عملية شراء صحيحة، فأنشئ الطلب بنجاح، وحُسب المبلغ الإجمالي، وانخفضت كمية المخزون بما يتطابق مع الكمية المطلوبة، كما تغيرت حالة الطلب إلى **Paid**.

أما الاختبار الثاني، فقد تضمن محاولة شراء كمية أكبر من الكمية المتوفرة. وعند اكتشاف نقص المخزون، رفع النظام الاستثناء **OutOfStockError**، وألغى جميع العمليات السابقة داخل المعاملة.

وبعد فحص قاعدة البيانات، تبين أن المخزون بقي كما كان قبل بدء العملية، ولم يُنشأ أي طلب جديد، مما يؤكد نجاح تطبيق مبدأ الذرية والمحافظة على اتساق البيانات.

9. اختبار الاستقرار تحت الضغط (Stress Testing)

لا يكفي أن يكون النظام صحيحاً من الناحية البرمجية، بل يجب التأكد من قدرته على الاستمرار في العمل تحت الأحمال المرتفعة، إذ تتعرض الأنظمة الحقيقية إلى آلاف الطلبات المتزامنة، وقد يؤدي ضعف التصميم أو سوء إدارة الموارد إلى انخفاض الأداء أو توقف الخدمة بالكامل.

ولذلك تُستخدم اختبارات الضغط لمحاكاة أعداد كبيرة من المستخدمين الحقيقيين، وقياس قدرة النظام على المحافظة على سرعة الاستجابة واستقرار الخدمة واكتشاف نقاط الضعف قبل نشر النظام في بيئة الإنتاج.

التطبيق في المشروع

لمحاكاة مئة مستخدم متزامن، موزعين إلى ثلاث فئات رئيسية تمثل الاستخدام الحقيقي للنظام **Locust** اعتمد المشروع على أداة

```
class ShopperUser(HttpUser):
```

tasks = [ShopperTasks]

weight = 70

class AdminUser(HttpUser):

tasks = [AdminTasks]

weight = 20

class BatchUser(HttpUser):

tasks = [BatchTasks]

weight = 10

تضمنت المحاكاة تنفيذ عمليات تصفح المنتجات، وإنشاء الطلبات، وتعديل المخزون، وتشغيل عمليات الدفعات، بصورة متزامنة طوال مدة الاختبار.

نتائج الاختبار الفعلي

استمر اختبار الضغط لمدة ستين ثانية، وخلال هذه الفترة استقبل النظام 2251 طلباً بمعدل 38.7 طلب في الثانية، بينما بلغ متوسط زمن الاستجابة 155.9 ميلي ثانية.

وسُجلت 243 حالة فشل بنسبة بلغت 10.8%، إلا أن التحليل التفصيلي أوضح أن معظم هذه الحالات لا تمثل أخطاء في النظام، وإنما كانت نتيجة مباشرة لآليات الحماية المطبقة داخله.

فقد كانت 154 حالة ناتجة عن الاستجابة 429 بسبب تجاوز الحد الأقصى لمحاولات تسجيل الدخول، بينما نتجت 72 حالة عن الاستجابة 403 نتيجة عدم توفر رمز المصادقة بعد رفض عمليات التسجيل السابقة، أما 17 حالة فقط فكانت نتيجة تعارض حقيقي في المخزون أثناء تنفيذ عمليات شراء متزامنة لمنتجات محدودة الكمية، وهي حالة متوقعة ومغطاة ضمن منطق النظام.

وتشير هذه النتائج إلى أن ما يقارب 93% من حالات الفشل المسجلة كانت ناتجة عن آليات حماية مقصودة، وليست أخطاء برمجية، الأمر الذي يؤكد قدرة النظام على العمل تحت الضغط مع المحافظة على سلامة البيانات واستقرار الخدمة.

10. القياس وتحديد الاختناقات (Benchmarking & Bottleneck Analysis)

يمثل القياس وتحليل الاختناقات (Benchmarking & Bottleneck Analysis) إحدى أهم مراحل تقييم أداء الأنظمة البرمجية، إذ لا يكفي ملاحظة أن النظام يعمل بصورة صحيحة، بل يجب تحديد الأجزاء التي تستهلك أكبر قدر من الزمن أو الموارد، لأنها تمثل العامل الأساسي الذي يحد من الأداء الكلي للنظام.

ويُقصد بالاختناق (Bottleneck) المكون أو المرحلة التي تستغرق زمناً أطول من بقية مراحل معالجة الطلب، بحيث يصبح تحسين أداء بقية الأجزاء غير ذي تأثير ملحوظ ما دام هذا الجزء لا يزال يمثل نقطة الإبطاء الرئيسية.

ولهذا تعتمد الأنظمة الحديثة على القياس الكمي لزمان الاستجابة قبل تطبيق أي تحسين وبعده، بهدف تقييم أثر ذلك التحسين بصورة موضوعية، بعيداً عن التقديرات النظرية أو الانطباعات الشخصية.

التطبيق في المشروع

لإجراء عملية القياس، تم إعداد برنامج مستقل يقوم بحساب زمن استجابة الطلبات بدقة باستخدام الدالة **time.perf_counter()**، والتي توفر قياساً عالي الدقة للفترات الزمنية القصيرة.

```
def timed_request(url):  
  
    start = time.perf_counter()  
  
  
    urllib.request.urlopen(  
  
        url,  
  
        timeout=5  
    )  
  
    return (  
  
        time.perf_counter() - start  
    ) * 1000
```

اعتمد الاختبار على إرسال مجموعة أولى من الطلبات قبل الاستفادة من التخزين المؤقت، ثم إرسال مجموعة ثانية بعد تفعيل الكاش، مع تسجيل متوسط زمن الاستجابة، والوسيط، وأعلى قيمة، وأقل قيمة لكل مجموعة، وذلك بهدف مقارنة الأداء بصورة دقيقة.

نتائج الاختبار الفعلي

أظهرت نتائج القياس القيم التالية:

المقياس	قبل التخزين المؤقت (5 طلبات)	بعد التخزين المؤقت (50 طلباً)
متوسط زمن الاستجابة	ms 2193.76	ms 2059.60
الوسيط	ms 2059.14	ms 2049.63
أعلى زمن استجابة	ms 2698.99	ms 2427.38
أقل زمن استجابة	ms 2044.10	ms 2015.69

من خلال هذه النتائج تبين أن نقطة الوصول الخاصة بعرض قائمة المنتجات كانت تمثل الاختناق الرئيسي قبل تفعيل التخزين المؤقت، حيث كانت جميع الطلبات تعتمد على تنفيذ استعلام مباشر إلى قاعدة البيانات.

وبعد تطبيق آلية التخزين المؤقت انخفض متوسط زمن الاستجابة بنسبة تقارب 6%، وهو تحسن فعلي يؤكد نجاح عمل الكاش، إلا أنه بقي أقل من التحسن النظري المتوقع في البيئات المثالية.

وتشير هذه النتيجة إلى وجود عامل تأخير ثابت داخل بيئة التشغيل، مثل زمن استجابة الشبكة المحلية أو وجود طبقة معالجة إضافية تسبق الوصول إلى الكاش، ولذلك جرى توثيق النتائج كما ظهرت فعلياً دون تعديل، حفاظاً على الدقة العلمية وموثوقية التقرير.

نتائج الاختبار الفعلي

أُجري اختبار عملي من خلال إرسال طلب إلى **ProductListView**، وقد أعاد النظام استجابة ناجحة برمز الحالة **200**، وفي الوقت نفسه سجل الديكوريتير معلومات الأداء داخل سجل النظام بالشكل التالي:

[AOP] GET ProductListView.get -> status=200 duration=2.49ms

كما أُجري اختبار آخر تضمن توليد استثناء داخل إحدى الدوال المغلفة، فتولى الديكوريتير تسجيل نوع الاستثناء وزمن التنفيذ، ثم أعاد رفع الاستثناء بصورة طبيعية دون التأثير في منطق التطبيق، مما يؤكد أن طبقة AOP تعمل بصورة مستقلة وشفافة.

نقاط التزامن المستخدمة داخل المشروع

الموقع	آلية التزامن	الغرض
middleware.py	threading.BoundedSemaphore	التحكم في عدد الطلبات المتزامنة المسموح بها
services.py	select_for_update() transaction.atomic()	قفل سجلات المنتجات أثناء تعديلها
services.py	cache.add() باستخدام Redis	منع تنفيذ عملية الدفع أكثر من مرة في الوقت نفسه
tasks.py	cache.add(BATCH_LOCK_KEY)	ضمان عدم تشغيل أكثر من دفعة معالجة في اللحظة نفسها

الخلاصة

نجح هذا المشروع في تطبيق جميع المتطلبات غير الوظيفية العشرة المطلوبة بصورة عملية داخل نظام تجارة إلكترونية متكامل، مع الاعتماد على تقنيات حديثة مثل Django و Django REST Framework و Redis و Celery.

وقد شملت هذه المتطلبات حماية البيانات من التعارض، وإدارة الموارد، وتنفيذ المهام غير المتزامنة، ومعالجة البيانات على دفعات، وتوزيع الأحمال، والتخزين المؤقت، وآليات التحكم في الأقفال، وسلامة المعاملات، واختبارات الضغط، وتحليل الأداء والاختناقات.

كما جرى تنفيذ طبقة مستقلة للبرمجة الموجهة للجوانب (AOP) لمراقبة الأداء، إضافة إلى محاكاة عملية لتوزيع الأحمال باستخدام خوارزمية Round-Robin، مع توضيح حدود هذه المحاكاة بما يتوافق مع نطاق المشروع.

واعتمدت جميع النتائج الواردة في هذا التقرير على تشغيل فعلي للنظام وإجراء اختبارات عملية في بيئة تنفيذ حقيقية، الأمر الذي يمنح النتائج مصداقية أكبر ويؤكد أن الاستنتاجات الواردة فيه مبنية على تطبيق عملي، وليس على افتراضات أو مناقشات نظرية فقط